Cryptography, Theory and Practice CIS 4930/6371 (Fall 2026)

Instructor: **Dr. Attila A. Yavuz**
TA: ***Sean Hernandez***
Contributor: ***Sean Hernandez***

November, 2026

Homework 6 (Due: November 16th, 2026)[100 Points]

(Upload your solutions to the CANVAS)
Requirements: HW groups are 2-3 people. Each student should contribute all parts of the answers. If one group member drops, the others are equally responsible (each HW can be handled by a single student if needed, so no excuse for missing a partner (if you are doing in group of 2), see syllabus for breaking-up with pair programming partner). Directly borrowing (e.g., copy-paste) from any material and putting in solutions (e.g, from online solutions, Wikipedia, or research papers) is plagiarism (see Syllabus for its corresponding actions). Please cite very carefully each resource you use but citing a solution does not give a license to directly put it as an answer. **All of your answers must be in your own words and interpretations. All your solutions will be passed through MOSS and Turnitin for plagiarism check.**
Note: Many of these questions require you to recall explanations/discussions during the class (this is why attendance is important), and also make research on the internet to find good answers. Please kindly note that very brief and/or informal explanations will potentially be not sufficient.
Remark: HW should be prepared by a text editor (e.g., Microsoft Word or Latex). Handwritten submissions are not accepted. In your answer key, please indicate the point of each question as it is done in this HW document. Preferably, take questions from here, plug them into your answer key with their sub scores, and provide your answer below them. This will facilitate the grading.
As indicated in Syllabus, **no partial credit is possible for coding answers that does not produce correct results with respect to test vectors, deviate from the description and rubric, are coded on platforms other than specified, or do not work on the grader's machine.**

---

1 Programming Assignment 1 – Post Quantum TLS: Kyber Key Encapsulation & Dilithium Signature [70 points]

This is a toy PQ-TLS (Post Quantum Transport Layer Security) exercise, which will simulate a Post Quantum TLS-like handshake. You will implement a key encapsulation mechanism (KEM) with Kyber and a digital signature scheme with Dilithium, using the liboqs library. The goal of this assignment is to understand the key encapsulation and decapsulation process, and to use Diithium to provide authentication when establishing a shared secret.
KEM is a public key crypto system that generates and encapsulates a secret key to be used for establishing secure communications. A sender, Alice, uses a public key to generate and encapsulate a secret key, and the reciever, Bob, user a private key to decapsulate and retrieve the secret key. Unlike some other public key crypto systems you have learned about, KEM only generates a secret key. This secret key can be used by the party to act as a symmetric key for authentication, and makes KEM a good primitive to use for creating hybrid crypto systems.

Implementation Setting

- You will use the liboqs library for all cryptographic operations.
- For key encapsulation, use the following specification in liboqs: `Kyber512`
- For Dilithium signing, use the following specification in liboqs: `Dilithium2`
- All cryptographic keys, ciphertexts, and signatures exchanged via files will be Hex-encoded strings.
- You are provided with a template that contains all necessary functions (**"PQTLSfunctions.c"**). These contain functions for reading/writing hex files and converting between byte arrays and hex strings. You may modify or add to these as needed.
- These provided functions are given to you to assist in your solution, but you are free to modify them, or use any functions of your choice as long as it produces the correct output. (Refer to the test vectors to understand the grading criteria)

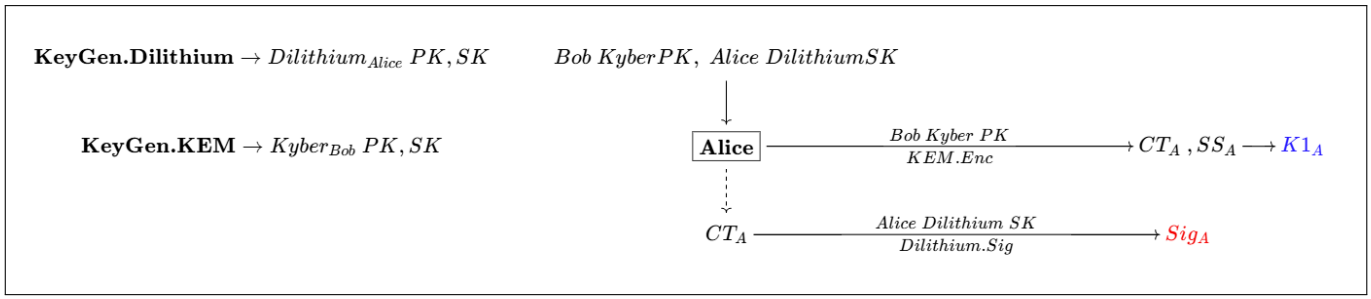
OFFLINE

For this assignment, assume the following key pairs have been pre-generated and distributed. Let sk denote a secret key and pk denote a public key

1. Alice is given Dilithium private/public key pair (sk_{DA}, pk_{DA}) as:
   `alice_dilithium_private.txt` (contains hex of sk_{DA})
   `alice_dilithium_public.txt` (contains hex of pk_{DA})
2. Bob is given Kyber private/public key pair (sk_{KB}, pk_{KB}) as:
   `bob_kyber_private.txt`(contains hex of sk_{KB})
   `bob_kyber_public.txt` (contains hex of pk_{KB})

A public key is derived from a secret key ($pk = KeyGen(sk)$), but liboqs generates them as a pair. You are provided 3 sets of Kyber and Dilithium key pairs for testing **(If you are curious or would like more key pairs for testing, you may generate your own using liboqs function, but this is not required)**.

Alice: Key encapsulation and Dilithium Signing [35 points]



1. Alice reads Bob's Kyber public key **bob_kyber_public.txt** from file. The key is in hex, so use the provided functions to read as Hex. (In real life, Alice gets Bob's public key from a certificate authority, but here, you just read it from a file!).
2. Alice reads her Dilithium private key **alice_dilithium_private.txt** from file. The key is in hex, so use the provided functions to read as hex.

- All key files contain Hex values. Use the provided functions to convert these Hex values into byte arrays (uint8_t).
- Verify that the length of the data read matches the expected key lengths defined by liboqs (OQS_KEM_kyber_512_length_public_key, OQS_SIG_dilithium_2_length_secret_key).

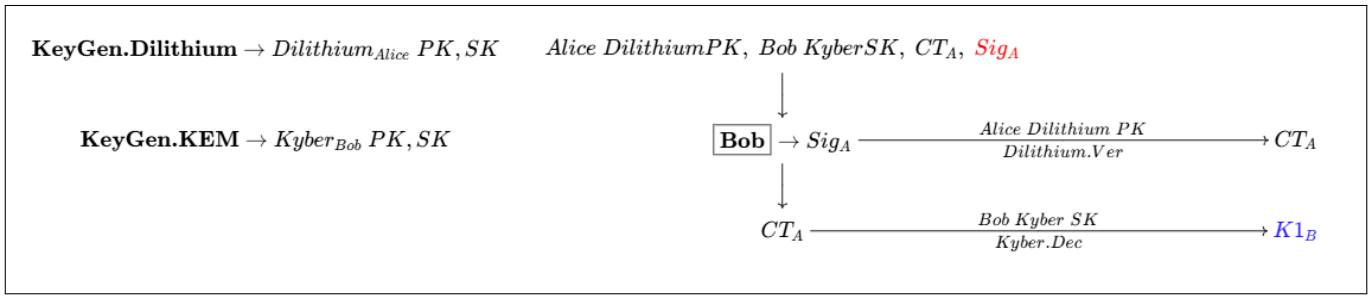
3. Using Bob's Kyber public key, Alice preforms key encapsulation. This can be represented as $(CT_A, K1_A) = Kyber.Encaps(pk_{k,b})$. This will generate:
   - A shared secret ($K1_A$)
   - A Kyber ciphertext (CT_A)

   Use the liboqs Kyber encapsulation functions.
4. After generating the shared secret ($K1_A$) and ciphertext (CT_A), Alice will use her Dilithium private key to sign the ciphertext. This can be represented as $\sigma_A = Dilithium.Sign(sk_{d,a})$. This will produce:
   - A Dilithium signature (σ_A)

   Use the liboqs Dilithium signature function.
5. After generating the Kyber ciphertext (CT_A), shared secret ($K1_A$), and Dilithium signature (σ_A), write each to file as:
   - Ciphertext: Convert to Hex and write to file called **kyber_ciphertext.txt**
   - Dilithium signature: Convert to Hex and write to file called **Dilithium_signature.txt**
   - Shared secret: Convert to Hex and write to file called **alice_shared_secret.txt**

   (Note: Alice is only writing the shared secret to file for testing purposes, so it can be compared with the result of Bob's decapsulation. It is not read or used by Bob in any way.)

Bob: Signature verification and Kyber decapsulation [35 points]



1. Bob reads Alice's Dilithium public key **(alice_dilithium_public.txt)** from file.
2. Bob reads his Kyber private key **(bob_kyber_private.txt)** from file
   - All key files contain Hex values. Use the provided functions to convert these Hex values into byte arrays (uint8_t).
   - Verify that the length of the data read matches the expected key lengths defined by liboqs (OQS_KEM_kyber_512_length_secret_key, OQS_SIG_dilithium_2_length_public_key).
3. Bob reads the Dilithium signature (σ_A) from **Dilithium_signature.txt**

4. Bob reads the Kyber ciphertext (CT_A) from **kyber_ciphertext.txt**
    - Both the signature and ciphertext are in Hex. Use the provided functions to convert these to byte arrays.
    - Verify the lengths of the ciphertext and signature match the expected lengths defined by liboqs (OQS_KEM_kyber_512_length_ciphertext, OQS_SIG_dilithium_2_length_signature).
5. Using Alice's Dilithium public key, verify the signature (σ_A).
    - If signature verification fails, Bob MUST abort the program, and should NOT proceed to decrypt the ciphertext.
    - If signature verification is successful, proceed to the next step
6. Decapsulate the Kyber ciphertext (CT) using Bob's Kyber private key **bob_kyber_private.txt** (sk_{KB}) to derive the shared secret ($K1_B$)
    - Decapsulation will produce the shared secret on Bob's side ($K1_B$).
    - Convert the shared secret to Hex string and write to file called **bob_shared_secret.txt**

Programming Assignment 1 - Specific Notes:

- Like ECDSA, Dilithium incorperates randomness into its scheme, meaning that signing the same message multiple times with the same key may produce different signatures. However, any valid signature for a given message and key must be verifiable with the corresponding public key.
- In a real-world scenario, public keys would typically be obtained from a trusted certificate authority. For this assignment, we simplify this by reading them directly from files.
- During grading, we will test with improper key and ciphertext sizes, as well as invalid and/or modified signatures. These will be chosen at random and your program must work.
- Invalid signature must cause Bob to exit the program. If Bob continues and decapsulates the ciphertext without verifying the signature (or verifying an invalid signature), you will lose credit.
- Compile your codes with the following commands:
  gcc alice.c -loqs -lssl -lcrypto -o alice
  gcc bob.c -loqs -lssl -lcrypto -o bob
- Run your codes for the first assignment with the following commands:
  ./alice bob_kyber_public.txt alice_dilithium_private.txt
  ./bob alice_dilithium_public.txt bob_kyber_private.txt
- A testing script **(VerifyPQTLS_P1.sh)** and a test vectors folder **(PQTLS_TestVectorsP1)** have been provided for you to use for testing your code. Please only submit your code if it is functional.
- In order to use the script, just put your codes (alice.c & bob.c), the rest of the provided files along with the "VerifyPQTLS.sh" in the same directory and run the following command in the terminal:
  bash VerifyPQTLS_P1.sh
- You may look at the testing script to understand how your code is being tested. This script is the same one used for grading on our side, however the test vectors will be different.
- **You must submit 2 files for this part of the assignment:**
  alice.c & bob.c

Rubric for Part 1:

- **Alice:**
  1. The **"kyber_ciphertext.txt"** file and the **"dilithium_signature.txt"** file **[20 pt]**
  2. The **"alice_shared_secret.txt"** file **[15 pt]**
- **Bob:**
  1. Correctly validating the signature and exiting upon invalid signature **[20 pt]**
  2. The **"bob_shared_secret.txt"** file **[15 pt]**

1.1 Programing Assignment 2 – Bidirectional KEM & XOR Key Derivation [20 points]

This is the second part of the Post Quantum TLS exercise. This assignment will have you implement a complete bidirectional key exchange where Bob will now perform key encapsulation and sign the ciphertext, Alice will authenticate, and both Alice and Bob will use the two keys to derive a final shared key using XOR operation and hash function. You will implement a key encapsulation mechanism (KEM) with Kyber and digital signature authentication with Dilithium, using the liboqs library.
In part 2, Bob will now use the appropriate keys to preform key encapsulation and a Dilithium signature. This will produce a shared secret ($K2_B$), and a ciphertext (CT_B) which he will sign. Now with knowledge of K1 and K2, Bob can preform an operation to derive the final shared secret, SS_{AB}. Alice will validate the signature and decapsulate the ciphertext to derive $K2_B$. With knowledge of K1 and K2, Alice can now preform an operation to derive the final shared secret SS_{AB}.

Implementation Setting

- You will use the liboqs and OpenSSL libraries for cryptographic operations.
- For key encapsulation, use the following specification in liboqs: `Kyber512`
- For Dilithium signing, use the following specification in liboqs: `Dilithium2`
- For hashing, use the following specification in OpenSSL: `SHA256`
- All cryptographic keys, ciphertexts, and signatures exchanged via files will be Hex-encoded strings.
- You are provided with a template that contains all necessary functions (**"PQTLSfunctions.c"**). These contain functions for reading/writing hex files and converting between byte arrays and hex strings. You may modify or add to these as needed.
- These provided functions are given to you to assist in your solution, but you are free to modify them, or use any functions of your choice as long as it produces the correct output. (Refer to the test vectors to understand the grading criteria)

OFFLINE

For this assignment, assume the following key pairs have been pre-generated and distributed. Let sk denote a secret key and pk denote a public key

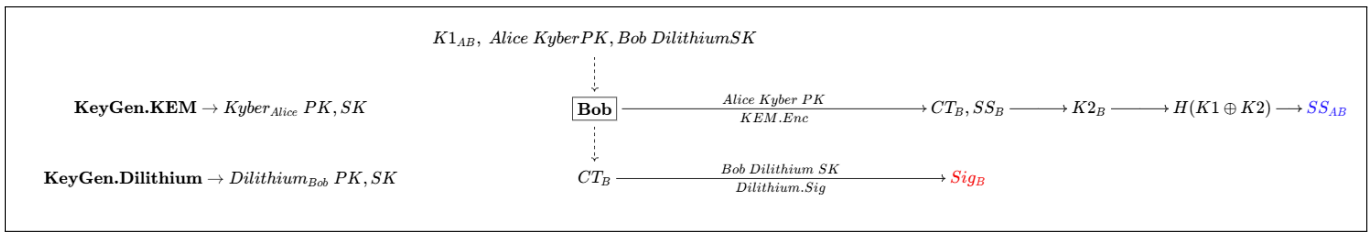
1. Alice is given Kyber private/public key pair (sk_{KA}, pk_{KA}) as:
   `alice_kyber_private.txt` (contains hex of sk_{KA})
   `alice_kyber_public.txt` (contains hex of pk_{KA})

2. Bob is given Dilithium private/public key pair (sk_{DB}, pk_{DB}) as:
   `bob_dilithium_private.txt` (contains hex of sk_{DB})
   `bob_dilithium_public.txt` (contains hex of pk_{DB})

   A public key is derived from a secret key ($pk = KeyGen(sk)$), but liboqs generates them as a pair. You are provided multiple sets of Kyber and Dilithium key pairs for testing.

Bidirectional KEM: Bob [10 pt]

Note: Part 2 will consist of 2 new files, alicexor.c and bobxor.c
Assume that $K1$ is already known by both Alice and Bob. $K1$ will be provided to you in a file named `k1_ab.txt`
Recall Part 1; Bob has just validated the signed ciphertext sent by Alice and has derived $K1$. At this point, both Alice and Bob have knowledge of $K1$. Now, Bob will preform KEM, sign the generated ciphertext, generate $K2_B$, and derive the final shared secret SS_{AB}



1. Bob reads $K1$ from file: `k1_ab.txt`
2. Bob reads Alice's Kyber public key **(alice_kyber_public.txt)** from file.
3. Bob reads his Dilithium private key **(bob_dilithium_private.txt)** from file
   - All key files contain Hex values. Use the provided functions to convert these Hex values into byte arrays (uint8_t).
   - Verify that the length of the data read matches the expected key lengths defined by liboqs (OQS_KEM_kyber_512_length_secret_key, OQS_SIG_dilithium_2_length_public_key).
4. Using Alice's Kyber public key, Bob preforms key encapsulation. This will generate:
   - A shared secret ($K2_B$)
   - A ciphertext CT_B

   Use the liboqs Kyber encapsulation functions.
5. After generating the shared secret ($K2_B$) and ciphertext (CT_B), Bob will use his Dilithium private key to sign the ciphertext. This will produce:
   - A Dilithum signature (σ_B)

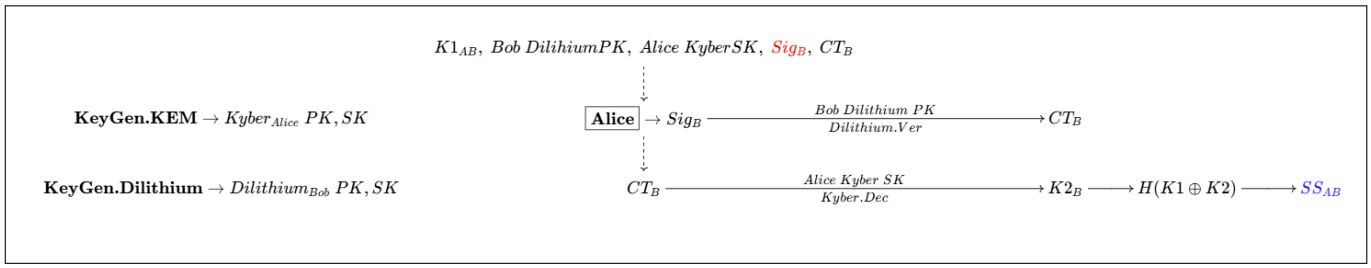
   Use the liboqs Dilithium signature function.

6. Now, with knowledge of $K1$ and $K2$, Bob can derive the final shared key, computed as: $H(K1 \oplus K2)$, where H is a hash function (use SHA-256).
   Compute the final shared key as: $H(K1 \oplus K2)$
   - XOR the two shared secrets $K1$ and $K2$
   - Apply SHA-256 hash function to the result
   - Write the result to file as `bob_final_key.txt`
7. After generating the Kyber ciphertext (CT_B), shared secret ($K2_B$), Dilithium signature (σ_B), and final key $H(K1 \oplus K2)$, write each to file as:
   - Ciphertext: Convert to Hex and write to file called **bob_kyber_ciphertext.txt**
   - Shared secret: Convert to Hex and write to file called **k2_bob.txt**
   - Signature: Convert to Hex and write to file called **bob_dilithium_signature.txt**
   - Final key: Convert to hex and write to file called **bob_final_key.txt**

   (Note: Bob is only writing his shared secret and final derived key to file for testing purposes, so it can be compared with the result of Alice's decapsulation and final key derivation. These 2 files should not be read by Alice in any way.)

Bidirectional KEM: Alice [10 pt]

Assume that $K1$ is already known by both Alice and Bob. $K1$ will be provided to you in a file named `k1_ab.txt`
To compute the final shared key, Alice needs to validate the signed ciphertext sent by Bob and decapsulate to derive $K2$.



1. Alice reads $K1$ from file: `k1_ab.txt`
2. Alice reads Bob's Dilithium public key **(bob_dilithium_public.txt)** from file.
3. Alice reads her Kyber private key **(alice_kyber_private.txt)** from file
   - All key files contain Hex values. Use the provided functions to convert these Hex values into byte arrays (uint8_t).
   - Verify that the length of the data read matches the expected key lengths defined by liboqs (OQS_KEM_kyber_512_length_secret_key, OQS_SIG_dilithium_2_length_public_key).
4. Alice reads the Dilithium signature (σ_B) from **bob_dilithium_signature.txt**
5. Alice reads the Kyber ciphertext (CT_B) from **bob_dilithium_signature.txt**
   - Both the signature and ciphertext are in Hex. Use the provided functions to convert these to byte arrays.

- Verify the lengths of the ciphertext and signature match the expected lengths defined by liboqs (OQS KEM kyber 512 length ciphertext, OQS SIG dilithium 2 length signature).

6. Using Bob's Dilithium public key, Alice verifies the signature (σ_B)
   - If signature verification fails, Alice MUST abort the program, and should NOT proceed to decrypt the ciphertext.
   - If signature verification is successful, proceed to the next step.
7. Decapsulate the Kyber ciphertext (CT_B) using Alice's own Kyber private key **alice_kyber_private.txt** to derive the shared secret ($K2$).
   - Decapsulation will produce the shared secret on Alice's side ($K2_A$)
   - Convert $K2_A$ to Hex string and write to file called **k2_alice.txt**
8. Now, with knowledge of $K1$ and $K2$, Alice can derive the final shared key, computed as: $H(K1 \oplus K2)$, where H is a hash function (use SHA-256).
   Compute the final shared key as: $H(K1 \oplus K2)$
   - XOR the two shared secrets $K1$ and $K2$
   - Apply SHA-256 hash function to the result
   - Write the result to file as `alice_final_key.txt`
9. After decapsulation, Alice derives the shared secret ($K2_B$) and computes the final key $H(K1 \oplus K2)$, write each to file as:
   - Shared secret: Convert to Hex and write to file called **k2_bob.txt**
   - Final key: Convert to hex and write to file called **alice_final_key.txt**

Programming Assignment 2 - Specific Notes:

- During grading, we will test with improper key and ciphertext sizes, as well as invalid and/or modified signatures. These will be chosen at random and must work.
- Invalid signature must cause Alice to exit the program. If Alice continues and decapsulates the ciphertext and derives the final key without verifying the signature (or verifying an invalid signature), you will lose credit.
- Compile your codes with the following commands:
  gcc alicexor.c -loqs -lssl -lcryopt -o alicexor
  gcc bobxor.c -loqs -lssl -lcryopt -o bobxor
- Run your codes for part 2 with the following commands:
  ./bobxor k1_ab.txt alice_kyber_public.txt bob_dilithium_private.txt
  ./alicexor k1_ab.txt bob_dilithium_private.txt alice_kyber_private.txt
  (for part 2, run Bob first, then run Alice)
- A testing script **(VerifyPQTLS_P2.sh)** and a test vectors folder **(PQTLS_TestVectorsP2)** have been provided for you to use for testing your code. Please only submit your code if it is functional.
- In order to use the script, just put your codes and the rest of the provided files along with the "VerifyPQTLS_P2.sh" in one folder and run the following command in the terminal:
  bash VerifyPQTLS_P2.sh
- You may look at the testing script to understand how your code is being tested. This script is the same one used for grading on our side, however the test vectors will be different.

- **You Must submit 2 files for this part of the assignment:**
  alicexor.c & bobxor.c

Rubric for Part 2:

- **Bob:**
  1. The **"bob_kyber_ciphertext.txt"** fine and the **"bob_dilithium_signature.txt"** file **[5 pt]**
  2. The **"k2_bob.txt"** file and the **"bob_final_key.txt"** file **[5 pt]**
- **Alice:**
  1. Correctly validating the signature and exiting upon invalid signature **[5 pt]**
  2. The **"k2_alice.txt"** file and the **"alice_final_key.txt"** file **[5 pt]**

1.2 Part 3 - Theory Written Response [20 points]

NIST PQC Standards

You have learned about Kyber, Dilihium, and FALCON. These are key encapsulation and digital signature schemes that have been standardized by NIST to meet post quantum standards. The algorithms previously mentioned have all been selected as winners by NIST on July 5th, 2022. Another hash based digital signature algorithm, SPHINCS+, was selected as a winner along with Kyber, Dilithium, and FALCON. In the next and fourth round of standardization, 4 more algorithms were chosen, with one of them being SIKE, a supersingular elliptic curve isogeny based key exchange algorithm, and another being HQC, a code-based public key encryption scheme.

Discuss the algorithms SPHINCS+, SIKE, HQC. What makes them quantum resilient? How do they differ from other similar, traditional cryptographic algorithms? Have any of them been broken, and if so, how?

Write your solution to **HW6_Group(number).pdf**

Overall Assignment Notes:

- Overall, you should be submitting 5 files total:
  - **alice.c and bob.c** for Programming Assignment 1
  - **alicexor.c and bobxor.c** for Programming Assignment 2
  - **HW6_Group(number).pdf** for Part 3
- You can refer to the liboqs library documentation `https://openquantumsafe.org/liboqs/` and the OpenSSL library documentation `https://www.openssl.org/docs/manmaster/man3/` to learn how to use the functions.
- We have provided test files and verification scripts for both assignments.
- In grading, different keys will be used and everything must work.
- Submit your code only if it is functional and passes the provided test cases.
- Code that throws errors or does not compile/run on our side receives zero credit.
- Follow all specific file names mentioned above, otherwise there will be 20% deduction for inconvenience.